

C.O.A.S.T. - Git and GitHub Cheat Sheet

Team: The Buoyz. Keep this handy. The two commands you use most: 'git pull' to start, and add/commit/push to save.

A. First-time setup on a NEW computer

Do this once per machine (a teammate getting started, or you on a second laptop). First install these once:

- Git: <https://git-scm.com/download/win> (accept defaults)
- Python 3.12: <https://www.python.org/downloads/windows/> and TICK 'Add python.exe to PATH'. Use 3.12 specifically (not 3.13/3.14).

Then get the project and set it up:

```
git clone https://github.com/<your-username>/CANTO.git
cd CANTO
py -3.12 -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
```

Note: If PowerShell blocks the activate line, run once: Set-ExecutionPolicy -Scope CurrentUser RemoteSigned (type Y), then retry.

Note: On Mac/Linux the venv lines differ: 'python3 -m venv .venv' then 'source .venv/bin/activate'.

B. Starting every work session

Run these each time you sit down, to get the latest code and turn on your environment (look for '(.venv)' in your prompt afterwards):

```
git checkout main
git pull
.\.venv\Scripts\Activate.ps1
```

C. Saving & pushing your changes (the constant one)

The cycle you run many times a day:

```
git add .
git commit -m "Short description of what you changed"
git push
```

- git add . = stage all your changes
- git commit = save a snapshot with a short, specific message
- git push = upload to GitHub so teammates can see it

D. Working with branches (once you're two programmers)

Start a new feature on its own branch:

```
git checkout main
git pull
git checkout -b feature/your-feature-name
```

Push that branch the first time (sets its GitHub home):

```
git push -u origin feature/your-feature-name
```

After that first push, plain 'git push' works for that branch. Then open a Pull Request on GitHub ('Compare & pull request') so a teammate reviews before it merges into main.

E. Getting a teammate's latest work into your branch

If main moved forward while you were working, pull it into your branch:

```
git checkout main
git pull
git checkout feature/your-feature-name
git merge main
```

F. Checking what's going on (safe, read-only - use often)

These never change anything; run them anytime to orient yourself:

```
git status          # what changed / staged / current branch
git branch           # list branches (* marks current)
git log --oneline -10 # last 10 commits
git diff             # exact lines changed but not committed
```

G. Common 'oops' fixes

```
git checkout -- <file> # discard uncommitted changes to ONE file
git switch main         # jump back to the main branch
git pull               # if push is rejected, pull first, then push
```

Note: If 'git push' is rejected, a teammate pushed first. Fix: run 'git pull', resolve any conflicts, then 'git push' again.

The two you'll use 95% of the time

- Start work: git pull
- Save work: git add . -> git commit -m "... " -> git push